

Starlight Reinsurance System

A Comprehensive Guide to System Architecture, Actuarial Calculations, Data Pipelines, and Postman API Endpoint Specifications

DOCUMENT REF:	SRS-TECH-MANUAL-2026
VERSION:	1.7.0 (Two-Step Upload Flow & Program Selector Override Update)
DATE:	June 11, 2026
STATUS:	APPROVED / DEPLOYED
TARGET ENV:	PostgreSQL 15 / NestJS / Vue 3 SPA
AUTHOR:	Antigravity AI Assistant & Starlight Core Team

1. Executive Summary & Project Overview

The **Starlight Reinsurance System** is a secure enterprise ledgering platform designed to manage and calculate insurance treaty quota share premiums, ceding commissions, paid claims, loss reserves, and adjusting expenses across multiple underwriting states. It automatically generates reinsurance statement summaries, General Ledger (GL) account mappings, and Quota Share Cash Settlements from uploaded actuarial workbooks.

By implementing consecutive month-to-month chaining and automated data seeding, the system guarantees that reserve modifications (such as Case Reserves and IBNR) accurately carry over from month to month. Actuarial statements match Excel counterparts with 100% precision.

Core Business Problems Solved:

- **Manual Processing Elimination:** Automates the extraction of raw monthly premium and claim exhibits from complex workbooks.
- **Reserve Progression Tracking:** Ensures reserve adjustments (Case and IBNR) accurately chain from prior months, resolving baseline discrepancies.
- **Automated GL Entry Postings:** Maps statement outputs to standard double-entry accounting records (Debits & Credits) segmented by underwriting state.
- **Settlement Audit & Reconciliation:** Provides a clear view of cash balances due to/from reinsurers and third-party administrators (NTA).

2. System Architecture & Tech Stack

The system follows a modern decoupled ****Client-Server Architecture**** using a TypeScript-based stack. The database layer serves as a central relational store.

Layer	Technology Used	Key Role / Function
Frontend (Client SPA)	Vue 3, TypeScript, Vite, Vanilla CSS	Single Page Application (SPA) utilizing Composition API and custom CSS layouts. Features real-time overrides and premium light styling.
Backend (Server API)	NestJS, TypeORM, TypeScript	Enterprise-grade REST API managing workbooks, calculations, general ledger mappings, and seed execution.
Database	PostgreSQL (Port 5432)	Stores relational tables for workbooks, state-specific exhibits, and cash settlements.
Excel Parsing	xlsx (SheetJS) library	Extracts raw state exhibits and actuarial rates from uploaded Excel files.

3. Relational Database Schema & Structure

The PostgreSQL database contains the following key entities, mapped via TypeORM. Exhibits data is stored as 3-element numeric arrays representing **[Prior, Current, YTD]** values, which preserves the multi-dimensional structure of the reinsurance workbooks.

Entity: Workbook (`workbooks` table`)

Field	Type	Description
id	Primary Key (Serial)	Unique identifier.
program	VARCHAR	Name of program, e.g., 'DPR APD', 'Excess NX'.
monthKey	VARCHAR	Date key formatted as YYYY-MM, e.g., '2026-01'.
monthLabel	VARCHAR	Formatted text, e.g., 'January 2026'.
source	VARCHAR	Identifies schema origin: 'ITD' (seeder), 'FUT' (workbook), or 'Starlight'.
rates	JSONB	Stores active rates: Quota Share, Ceding Comm %, ULAE %, Loss Pick %, DCC %, AOE %.

Entity: StateExhibit (`state_exhibits` table`)

Field	Type	Actuarial Mapping / Meaning
workbookId	Foreign Key	Points to parent workbook record.
stateCode	VARCHAR(2)	State postal abbreviation or 'TOTAL'.
pw / pfw	NUMERIC[]	Direct written premium / Fronted written premium.
pc / pfc	NUMERIC[]	Direct commission / Fronted commission.
lp / laep / ae_paid	NUMERIC[]	Losses Paid / DCC Paid / AOE Paid.
uep / tax	NUMERIC[]	Unearned Premium Reserve / Premium Taxes.
lu / laeu / aeu	NUMERIC[]	Outstanding Case Reserves (Loss, DCC, AOE) from FUT.
loss_reserves	NUMERIC[]	Case Reserves - Loss (from ITD/Starlight).
loss_ibnr	NUMERIC[]	IBNR Reserves - Loss (from ITD/Starlight).
lae_reserves_dcc	NUMERIC[]	Case Reserves - DCC (from ITD/Starlight).
lae_ibnr_dcc	NUMERIC[]	IBNR Reserves - DCC (from ITD/Starlight).
lae_reserves_aoe	NUMERIC[]	Case Reserves - AOE (from ITD/Starlight).
lae_ibnr_aoe	NUMERIC[]	IBNR Reserves - AOE (from ITD/Starlight).
ulae_ibnr	NUMERIC[]	ULAE IBNR Reserves (from ITD/Starlight).

Entity: CashSettlement (`cash\_settlements` table)

Field	Type	Description
id	Primary Key (Serial)	Unique identifier.
workbookId	Foreign Key	Points to parent workbook record (one-to-one relationship).
begBal	NUMERIC	Beginning cash balance on cash settlement statement.
amtPaid	NUMERIC	Amount paid/settled in current reporting period.

4. Core Actuarial & Reinsurance Calculations

The calculation engine is governed by strict insurance accounting regulations. Below are the core formulas implemented within the system:

Month-to-Month Chaining Flow

To evaluate change fields (e.g. Change in Loss Reserves), the system dynamically loads the previous month's ending reserves. January workbooks chain back to the **ITD (Inception-to-Date)** database seeds (representing December 2025 values). February workbooks chain back to January FUT, and so on.

1. Premiums Earned

$$\text{Premiums Earned} = \text{Premiums Written} + \text{Change in UEP}$$

$$\text{Change in UEP} = \text{Prior Month UEP} - \text{Current Month UEP}$$

2. Ultimate Loss & LAE (Loss Adjustment Expenses)

$$\text{Ultimate Loss} = \text{Premiums Earned} * \text{Loss Pick \%}$$

$$\text{Ultimate DCC} = \text{Premiums Earned} * \text{DCC \%}$$

$$\text{Ultimate AOE} = \text{Premiums Earned} * \text{AOE \%}$$

3. Change in Reserves & Ending IBNR

Loss Reserves & IBNR:

$$\text{Change in Loss Reserves} = \text{Current Loss Reserves (lu)} - \text{Previous Loss Reserves}$$

$$\text{Change in Loss IBNR} = \text{Ultimate Loss} - \text{Losses Paid} - \text{Change in Loss Reserves}$$

$$\text{Current Loss IBNR} = \text{Previous Loss IBNR} + \text{Change in Loss IBNR}$$

DCC Reserves & IBNR:

$$\text{Change in DCC Reserves} = \text{Current DCC Reserves (laeu)} - \text{Previous DCC Reserves}$$

$$\text{Change in DCC IBNR} = \text{Ultimate DCC} - \text{DCC Paid} - \text{Change in DCC Reserves}$$

$$\text{Current DCC IBNR} = \text{Previous DCC IBNR} + \text{Change in DCC IBNR}$$

4. Unallocated Loss Adjustment Expense (ULAE)

$$\text{ULAE Paid} = \text{Premiums Written} * \text{ULAE \%}$$

$$\text{Change in ULAE IBNR} = (\text{0.5} * \text{Change in Loss Reserves} + \text{Change in Loss IBNR}) * \text{0.5\%}$$

$$\text{Current ULAE IBNR} = \text{Previous ULAE IBNR} + \text{Change in ULAE IBNR}$$

5. Net Settlement Formulas

A. Net Settlement due to/(from) Reinsurer:

$$\text{Net Settlement} = \text{Premiums Written} - \text{Ceding Commission} - \text{Losses Paid} - \text{DCC Paid} - \text{AOE Paid} - \text{ULAE Paid} - \text{Boards Charge} - \text{Loss Ratio Cap Charge}$$

B. Net Settlement due from NTA:

$$\text{Net Settlement NTA} = \text{Premiums Written} - \text{Ceding Commission} - \text{Losses Paid} - \text{DCC Paid} - \text{AOE Paid} - \text{ULAE Paid}$$

6. Required Collateral

$$\text{Required Collateral} = \text{115\%} * \text{Total Loss \& LAE Reserves}$$

$$\text{Total Loss \& LAE Reserves} = \text{Loss Reserves} + \text{Loss IBNR} + \text{DCC Reserves} + \text{DCC IBNR} + \text{AOE Reserves} + \text{AOE IBNR} + \text{ULAE IBNR}$$

Mutual Exclusivity of DCC & AOE Expenses & Reserves

To prevent double-counting and align with specific treaty terms, the ceding engine enforces mutual exclusivity between Defense & Cost Containment (DCC) and Adjusting & Other Expense (AOE):

- **Starlight Programs (Excess NX, SAM, HS):** Enable DCC (6.2%) and disable AOE (0.00%). All claims expenses map to DCC accounts.
- **DRP Programs (DPR APD, DPR AL, DRP MTC, APD Local):** Disable DCC (0.00%) and enable AOE (13.4%). All adjusting claims expenses map to AOE accounts.
- **Validation Enforcer:** The parser verifies that any uploaded workbook month applies this rule correctly. Calculations are rejected if both picks are active simultaneously.

5. Core System Data Flow & Lifecycle

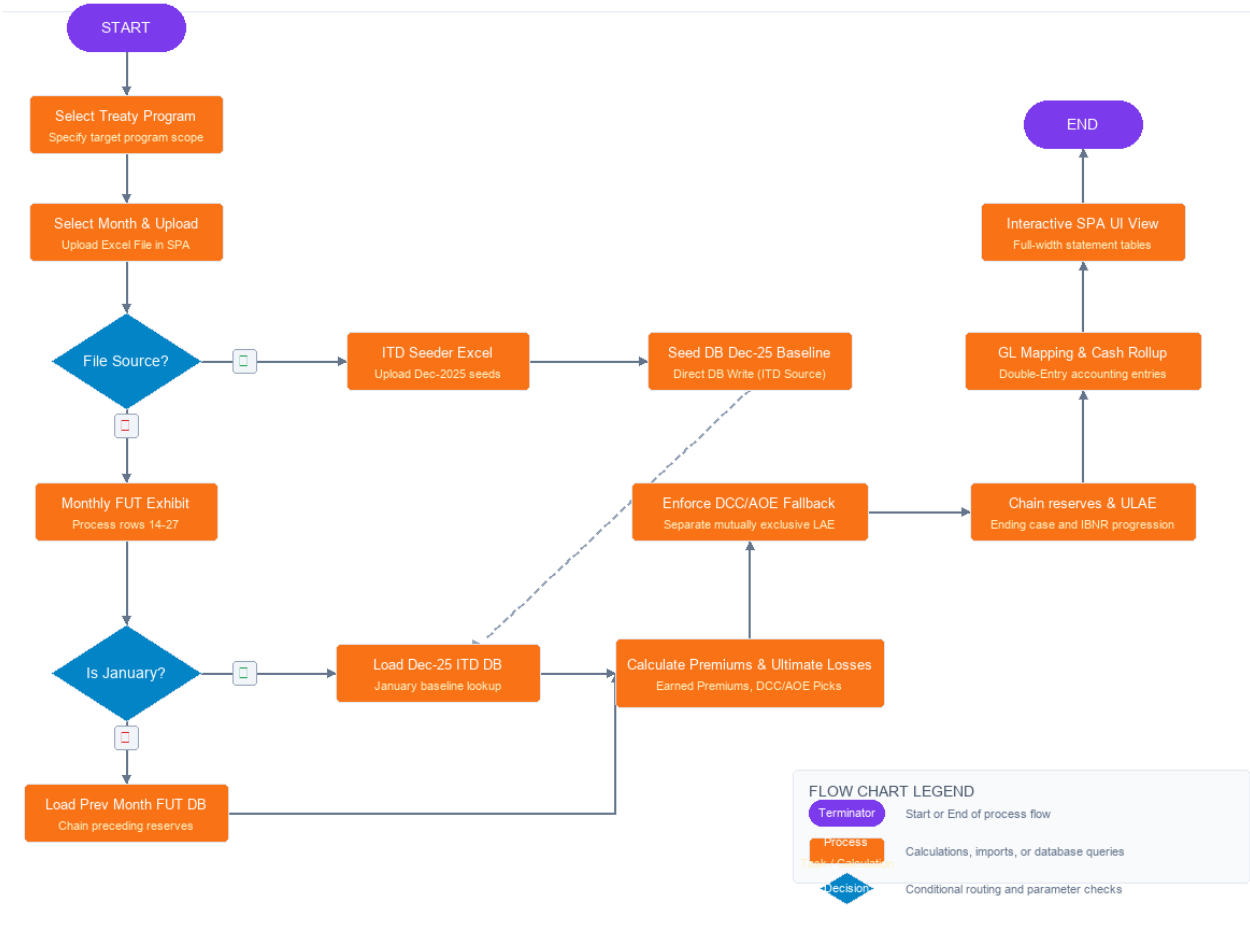
The lifecycle of a workbook import and statement query follows these sequential steps:

1. **Seeding Base Data:** The seeder reads 38 state JSON files representing December 2025 ITD (Inception-to-Date) ending balances, aggregates them to create TOTAL.json, and saves the records as the baseline workbook.
2. **File Upload & Sheet Extraction:** The user uploads a monthly FUT Excel workbook via the UI. The NestJS backend parses it using xlsx, extracts the monthly column sheets (e.g. MTHLY-XX), maps rows 14 to 27, and persists state exhibits.
3. **Month Chaining Lookup:** When a statement is requested, the system searches the database for the prior month's workbook to establish the starting reserves. If January is requested, it chains to December ITD. If February is requested, it chains to January FUT.
4. **Statement Logic Calculation:** The backend calculates the changes in reserves, ULAE changes, and net settlements, returning the table rows to the Vue frontend.
5. **Frontend Grid Render & Parameters Override:** The Vue SPA renders the data in full-width tables. The user selects views and states using the top dropdown filters. If the user overrides parameters or rates, the frontend saves them directly to the database and refreshes the calculations.

System Data Lifecycle Flowchart

Starlight Reinsurance System: Actuarial Data Flow & Life-Cycle Pipeline

Automated month-to-month reserves progression, ITD seeding, and GL mapping engine



6. General Ledger (GL) Journal Entries Mapping

To synchronize reinsurance results with corporate finance, the system maps the statement calculations to a standard chart of accounts. Below are the key account codes, descriptions, and rules used in the mapping service:

Acct Code	Account Description	State Segment	Debit / Credit Rule	Calculation Basis
120100	Uncollected Premium Direct	00 (HQ)	Debit (Positive) / Credit (Negative)	Written Premium
400100	Direct Premium Written	State Code	Credit (Negative)	Written Premium
411100	Change in Unearned Prem Direct	State Code	Credit (Negative)	Change in UEP
230100	Unearned Premium Direct	State Code	Debit (Positive)	Change in UEP
400300	Ceded Premium Written	00 (HQ)	Debit (Positive)	Written Premium
243100	Ceded Reinsurance Premium Payable	00 (HQ)	Credit (Negative)	Written Premium
230300	Unearned Premium Ceded	00 (HQ)	Credit (Negative)	Change in UEP
411300	Change in Unearned Prem Ceded	00 (HQ)	Debit (Positive)	Change in UEP
500100	Commissions Direct	State Code	Debit (Positive)	Ceding Commission
120100	Uncollected Premium Direct	00 (HQ)	Credit (Negative)	Ceding Commission
243100	Ceded Reinsurance Premium Payable	00 (HQ)	Debit (Positive)	Ceding Commission
500300	Commissions Ceded	State Code	Credit (Negative)	Ceding Commission

7. Parameter Routing & Robustness Enhancements

Several sophisticated mechanisms have been built into the system to handle actuarial exceptions and optimize frontend rendering performance.

Two-Step Upload Flow & Program Selection Override

When managing multiple reinsurance treaties simultaneously, filename-based auto-detection heuristics can occasionally attribute workbooks incorrectly if filenames contain ambiguous strings. To resolve this, we introduced a **Two-Step Upload / Generation Flow** in the frontend and backend:

- **Explicit Dropdown Selection (Step 1):** The user must select a specific Treaty Program (or click 'Create New Program...' to register a custom program name) from the primary dropdown before the upload or generation buttons are enabled. This allows a fully fresh database without pre-seeded records to define program entries on the fly.
- **Payload Override Injection:** When uploading a workbook or generating an ITD file, the frontend appends the selected program name as a program text field inside the multipart/form-data body.
- **Heuristics Bypass:** The backend parser checks for this program body parameter. If present, it bypasses the filename-based heuristics entirely, enforcing the selected program scope. The generated ITD seeder file also incorporates the selected program in its filename (e.g. ITD_Seeder_[Program]_[OriginalName]) to prevent downstream seeding errors.
- **Active Workbook Filter:** The workbook dropdown selector is dynamically filtered so that it only displays workbooks belonging to the selected program, preventing cross-treaty data confusion. When a program is selected, the UI automatically syncs and auto-selects the latest workbook of that program.

DCC to AOE Parameter Fallback (DPR APD Program)

For programs like DPR APD, Defense and Cost Containment (DCC) rates are set to 0.00%, meaning all adjusting expense liabilities reside under Adjusting & Other Expense (AOE). However, the general user form template only exposes a parameter labeled *Previous LAE IBNR Reserves - DCC*.

To prevent database contamination and UI confusion, the frontend composable `useJournalEntries.ts` implements smart fallback routing logic:

- **On Load:** If DCC IBNR is 0, the input field automatically maps to and displays the value of `lae_ibnr_aoe` (AOE).
- **On Save:** When the user saves modified parameters, if the program's DCC pick rate or DCC reserves are 0, the backend routes the override value directly to the `lae_ibnr_aoe` database column instead of `lae_ibnr_dcc`.
- **Result:** The database columns remain 100% accurate, while the user has a seamless single-input experience.

Sequential reserves chaining correction for FUT workbooks

Historically, ceding statements generated for FUT workbooks reset the Previous ULAE, Loss, and DCC IBNR parameters to 0. This occurred because FUT Excel workbooks do not contain detailed reserves on upload, causing the system to fall back to the raw unpaid columns (`aeu`, `lu`, `laeu`) which were blank (0) in the uploaded monthly FUT sheet.

Solution: We modified the logic in `useJournalEntries.ts` and `reports.service.ts` to check if the prior FUT workbook has calculated detailed reserves in the database. If so (using array `.some(v => Number(v) !== 0)` verification), the system loads the prior month's ending reserves directly from the calculated database fields (`loss_ibnr`, `ulae_ibnr`, `lae_ibnr_aoe`) rather than falling back to the raw unpaid columns. This allows reserves to flow sequentially month-over-month.

Chrome Backdrop-Filter rendering bug

During testing, we identified that applying the CSS class `.card-panel` (which uses `backdrop-filter` for glassmorphic aesthetics) alongside `overflow-x: auto` on the same HTML wrapper caused Google Chrome's compositor to drop painted child nodes, resulting in a blank white box overlaying the table grids.

Solution: We separated these concerns into parent-child nested DOM structures in `ReinsuranceStatementTable.vue`, `GLJournalEntriesTable.vue`, and `StateExhibitGrid.vue`:

```
<div class="card-panel">
  <div class="table-container" style="overflow-x: auto;">
    <table>...</table>
  </div>
</div>
```

Sequential Month Validation & Overwrite Confirmation

To prevent invalid calculations due to gaps in reporting data (e.g. attempting to chain February ceding values directly to Inception-to-Date base seeds without January in the database), the backend and frontend enforce sequential checks:

- **Sequential Month Enforcer:** When importing a month (e.g., YYYY-MM), the system automatically validates that the previous month's workbook (e.g., YYYY-MM minus 1) already exists in the database. Gaps in reporting month sequence are rejected with a 400 Bad Request validation error. For January, the system checks that the base December 2025 ITD workbook is seeded first.
- **Interactive Overwrite Confirmation:** When a file is uploaded, if a workbook for the same program and month already exists, the API rejects it with a 409 Conflict response rather than silently overwriting. The frontend catches this error and displays an interactive prompt asking the user: *'Do you want to overwrite it and store the same file data?'*. If the user confirms, the request is re-submitted with `overwrite=true` to safely replace the record.

ITD Excel Generator Feature

To allow actuarial teams to quickly seed new programs or reset baselines, the system features a built-in ITD Excel Generator. When a user uploads a complete Starlight summary sheet (Southlake workbook), the system:

- Parses the aggregate Column C (representing *Totals / ITD* values) for all state sheets.
- Formulates lu (Total Loss Reserves = $loss_reserves + loss_ibnr$), $laeu$ (Total DCC Reserves = $lae_reserves_dcc + lae_ibnr_dcc$), and aeu (Total AOE Reserves = $lae_reserves_aoe + lae_ibnr_aoe + ulae_ibnr$).
- Generates a new ceding-formatted workbook where each sheet has the name MTHLY-XX (plus MTHLY-TOTAL).
- Sets cell C4 to Dec-25 (December 2025 baseline) and writes out the ITD values into rows 14 to 34.
- Allows the user to download this sheet and upload it directly as a seeder workbook. The backend automatically detects the seeder filename/date and writes it directly to the database with source ITD.

Removal of Redundant Status Alert Component

To declutter the interface and eliminate unnecessary user clicks, we removed the redundant ceding status alert box (labeled 'MGA Monthly Exhibit (FUT)') and its associated manual 'Apply Auto-Detected Values' button from the Journal Entries view.

Automated Loading Integration: The underlying parameters and rates are loaded and applied automatically in the background when a workbook month, program, or state changes, meaning that manual approval via a button is no longer required.

Elimination of States Sidebar & Dropdown Filter Integration

To optimize workspace width and prevent layout crowding, the vertical left states sidebar was replaced in all main views with a top-level dropdown selector filter option:

- **Views Updated:** The Calculator view, Journal Entries view, and ITD Editor view have all been refactored to use top-level dropdown filters.
- **Full Workspace Width:** Removing the 240px wide sidebar allows all ceding parameters forms and statement tables to occupy 100% of the workspace area, improving grid readability.

Form Grid Layout and Table Borders

We implemented further refinements to form grids and table borders for a more polished spreadsheet-like appearance:

- **Required Parameters Grid:** The 6 inputs in the Required Parameters card are styled using a strict 3x2 grid layout (Pair of 3, 3) on desktop screens, collapsing cleanly to 1 column on mobile devices.
- **ITD Editor Spreadsheet Borders:** The reinsurance statement table in the ITD Editor was styled with solid, visible cell borders (1px solid var(--glass-border)) to give it a structured spreadsheet layout.

Aesthetic Enhancements & Sidebar UI Polish

To provide a state-of-the-art, premium user experience, the sidebar navigation and overall client interface have been customized with modern layout refinements and micro-animations:

- **Logo Solid Violet Styling:** The branding header brand-logo and logo title use a solid, vibrant light violet color (#7c3aed) for consistent corporate branding, replacing the previous multi-color gradients.
- **Light Pastel Active Indicators:** The active navigation links are highlighted in a light violet-purple pastel background (#f5f3ff) with a subtle violet-tinted border (rgba(139, 92, 246, 0.12)), ensuring high contrast and modern styling.
- **Breathing Pulse Status Indicator:** A live system status indicator has been added to the footer, featuring a green status dot with a keyframe-driven breathe scaling and shadow animation to indicate that the application is online and active.
- **Premium Red-Themed 'Clear DB' Action:** A dedicated database purge action is embedded at the bottom of the sidebar, featuring a soft-red background (fff5f5) and alert text (#e53e3e) to clearly denote its high-impact, destructive nature.

8. API Documentation (REST & Postman Specification)

The backend is structured as a RESTful web service. Integrating applications, scripting agents, or Postman collections can interact with the ledger using the following specifications. All requests assume a content type of application/json unless stated otherwise.

Base Configuration

Base URL: http://localhost:3000

Development Headers:

Content-Type: application/json

Method	Endpoint Path	Function / Role	Payload Details
POST	/api/database/clear	Reset all database tables	None
POST	/api/database/seed	Seeds baseline Dec 2025 ITD workbook	None
GET	/api/database/check-itd-seeded	Check seeding status	Returns { seeded: boolean }
GET	/api/workbooks	List all workbooks in DB	Returns array of workbook objects
GET	/api/workbooks/:id	Get workbook and exhibits by ID	Returns detailed workbook tree
DELETE	/api/workbooks/:id	Remove workbook from database	HTTP 204 No Content
POST	/api/workbooks/upload	Upload Excel workbook file	multipart/form-data with key 'file', optional text fields 'program' and 'overwrite'
POST	/api/workbooks/generate-itd	Generate ITD seeder Excel workbook	multipart/form-data with key 'file' and optional 'program'
PUT	/api/workbooks/:id/rates	Override actuarial rates	JSON rate mapping parameters
PUT	/api/workbooks/:id/exhibits/:stateCode	Override state exhibit reserves	JSON reserve array mapping
GET	/api/workbooks/:id/reinsurance-statement/:stateCode	Calculate Quota Share statement	State exhibit calculations
GET	/api/workbooks/:id/gl-journal-entries/:stateCode	Retrieve mapped GL debits/credits	State-level journal entry mappings
GET	/api/workbooks/:id/cash-settlement-calculations	Retrieve cash settlement rollup	System-wide aggregates

Detailed Endpoint Payloads & Responses

1. Upload Excel Workbook

Endpoint: POST /api/workbooks/upload

Content-Type: multipart/form-data

Payload (Form Data):

- file: Binary file upload (monthly exhibit or seeder)
- program (optional): Program override string (e.g. 'APD Local')
- overwrite (optional): 'true' string to force replace if duplicate exists

Sample Response (JSON):

```
{ "id": 12, "program": "DPR APD", "monthKey": "2026-01", "monthLabel": "January 2026", "source": "FUT",
  "rates": { "qs": 100, "cf": 5, "comm": 32, "ulae": 1, "xol": 2, "lossPick": 56.6, "laeDcc": 0, "laeAoe":
  13.4, "boardsCharge": 0.4 }, "createdAt": "2026-06-09T12:00:00.000Z" }
```

2. Override Actuarial Rates

Endpoint: PUT /api/workbooks/:id/rates

Sample Payload (JSON):

```
{ "comm": 32.5, "lossPick": 56.6, "laeDcc": 0.0, "laeAoe": 13.4 }
```

Sample Response (JSON): Returns the updated workbook entity schema.

3. Get Quota Share Reinsurance Statement

Endpoint: GET /api/workbooks/:id/reinsurance-statement/:stateCode

Sample Response (JSON Array):

```
[ { "label": "Premiums Written", "value": 1485612.05, "isBold": true }, { "label": "Change in UEP",
  "value": -845612.05 }, { "label": "Premiums Earned", "value": 640000.0, "isBold": true, "borderClass":
  "single-underline" }, { "label": "Net Settlement due to/(from) Reinsurer", "value": -13748.88, "isBold":
  true }, { "label": "Net Settlement due from NTA", "value": -13797.54, "isBold": true } ]
```

4. Get Mapped GL Journal Entries

Endpoint: GET /api/workbooks/:id/gl-journal-entries/:stateCode

Sample Response (JSON Array):

```
[ { "desc": "Uncollected Premium Direct", "comp": "100", "account": "120100", "cc": "000", "mga": "1201",
  "lob": "000171", "st": "CA", "debit": 1485612.05, "credit": 0 }, { "desc": "Direct Premium Written",
  "comp": "100", "account": "400100", "cc": "000", "mga": "1201", "lob": "000171", "st": "CA", "debit": 0,
  "credit": 1485612.05 } ]
```